MODULE 5

Design the schema

GroupBy & pagination · DbContext · Fluent API · one-to-many · join entity & OnDelete

Reporting

```
await context.Enrollments
  .GroupBy(e => e.CourseId)
  .Select(g => new {
    g.Key,
    Count = g.Count()
  })
  .ToListAsync(ct);
```

From questions to professional reports

Session outcomes

By the end of this session, you will:

- Aggregate with GroupBy and Select so SQL does the math
- Paginate with Skip and Take for large TMS lists
- Configure DbContext for clarity and team conventions
- Use Fluent API to override defaults without data annotations on entities
- Split mapping into IEntityTypeConfiguration classes so OnModelCreating stays short
- Define one-to-many and many-to-many-with-payload with explicit OnDelete you can defend

Grouping let the database count

GroupBy and projection

Ask the room

You need counts per course. Do you load every enrollment into C# and loop?

```C#
var courseStats = await context.Enrollments
  .GroupBy(e => e.CourseId)
  .Select(g => new {
      CourseId = g.Key,
      StudentCount = g.Count(),
      AverageGpa = g.Average(e => e.Student.GPA)
    })
```

SQL shape (conceptual)

SELECT CourseId, COUNT(*), AVG(...) ... GROUP BY CourseId one query; only aggregates cross the wire.

Pagination Skip, Take, OrderBy

```C#
int pageSize = 25, pageNumber = 2;

var page = await context.Students

  .OrderBy(s => s.Name)   // required

  .Skip((pageNumber - 1) * pageSize)

  .Take(pageSize)

  .ToListAsync(ct);
```

Golden rule

Always paginate public lists. Without OrderBy before Skip/Take, row order is undefined users see rows jump or vanish between pages.

TmsDbContext the conversation

C# 14

```
public class TmsDbContext(
  DbContextOptions<TmsDbContext> options)

  : DbContext(options)
{

  public DbSet<Student> Students => Set<Student>();

  public DbSet<Course> Courses => Set<Course>();

  public DbSet<Enrollment> Enrollments => Set<Enrollment>();

  protected override void OnModelCreating(ModelBuilder b)
  {

    b.ApplyConfigurationsFromAssembly(

      typeof(TmsDbContext).Assembly);
```

Key concept

DbSet is the noun (table-shaped entry point). DbContext is the conversation queries and SaveChanges flow here.

Fluent API CourseConfiguration

Fluent

```
public class CourseConfiguration

  : IEntityTypeConfiguration<Course>
{

  public void Configure(EntityTypeBuilder<Course> b)
  {

    b.HasKey(c => c.Id);

    b.Property(c => c.Title).IsRequired().HasMaxLength(200);

    // Relationship detail on the next slides
```

- Convention Id as key, etc.
- Data annotations [Required], [MaxLength] on entities
- Fluent API preferred here: mapping stays out of M1 domain types

One-to-many · explicit delete policy

Fluent

```
public class CourseConfiguration

  : IEntityTypeConfiguration<Course>
{

  public void Configure(EntityTypeBuilder<Course> b)
  {

    b.HasMany(c => c.Enrollments)

      .WithOne(e => e.Course)

      .HasForeignKey(e => e.CourseId)

      .OnDelete(DeleteBehavior.Restrict);
```

- Foreign key: CourseId in SQL
- Navigation: dot from Enrollment to Course
- Restrict on Course→Enrollment: cannot delete a course that still has enrollments (canonical TMS defence)

Join entity · Student ↔ Enrollment ↔ Course

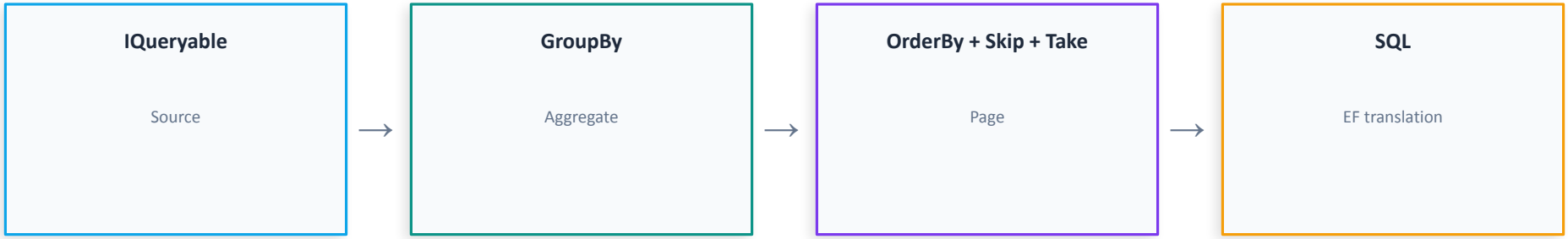
Fluent

```
public class EnrollmentConfiguration

  : IEntityTypeConfiguration<Enrollment>
{

  public void Configure(EntityTypeBuilder<Enrollment> b)
  {

    b.HasKey(e => e.Id);

    b.HasIndex(e => new { e.StudentId, e.CourseId }).IsUnique();


    b.HasOne(e => e.Student).WithMany(s => s.Enrollments)
       .HasForeignKey(e => e.StudentId);

    b.HasOne(e => e.Course).WithMany(c => c.Enrollments)
       .HasForeignKey(e => e.CourseId);
```

Why not List<Course> on Student?

Enrollment carries Grade, dates, status—payload on the bridge, not a dumb link table.

Blueprint report shape in SQL



Teaching cue

OrderBy before Skip/Take unordered sets make pagination lie.

Blueprint · TMS graph

Student

One ⟶ many Enrollments

Enrollment

Join entity: grade, dates, payload on the link

Course

One ⟶ many Enrollments

Teaching cue

OrderBy before Skip/Take; OnDelete explicit—provider defaults differ and bite in production.

Lab brief Session 2

- Lab handout **`M5-Lab-Session-2.md`** — Exercises **3 + 4 + 5**
- Build: paginated student list, top-5 course report in SQL, three `IEntityTypeConfiguration` classes, OnDelete you can defend
- Done when: SQL shows LIMIT/OFFSET for paging and GROUP BY for top-5; `OnModelCreating` only calls `ApplyConfigurationsFromAssembly`
- Next session: **`M5-Session-3-Presentation.md`** — operate the schema (migrations, N+1, bulk, filters, Repository/UoW write-up)

Session summary

Topic	Takeaway
GroupBy	Counts and averages in SQL, not in-memory loops
Skip / Take	Pagination engine — never ship full tables to the client
OrderBy	Mandatory before Skip/Take for stable TMS pages
DbContext	Single entry point; DbSet for each aggregate root / table
Fluent API	Precise schema control without polluting domain models
FK + OnDelete	Glue and delete policy explicit for Course→Enrollment and related paths
Join entity	Enrollment carries grade/payload — not a hidden many-to-many

What this looks like in an interview

Pagination: OrderBy before Skip/Take. Relationships: defend Restrict vs Cascade for at least one TMS direction (e.g. deleting a Course with active enrollments). Tie joins to Enrollment.Grade payload.

Next: Session 3 · Operate the schema

Migrations as code review, N+1 demonstrated and fixed, AsNoTracking, ExecuteUpdateAsync, soft-delete filters, Tier 6 Repository/UoW summary — then M6 REST.